

SMART CONTRACT CODE REVIEW AND SECURITY ANALYSIS REPORT

This document may contain confidential information about IT systems and the intellectual property of the customer as well as information about potential vulnerabilities and methods of their exploitation.

The report containing confidential information can be used internally by the customer or it can be disclosed publicly after all vulnerabilities fixed - upon a decision of the customer.

Document

Name	Smart Contract Code Review and Security Analysis Report for CHAIN GAMES
Type	A contract to manage bets placed in CHAIN GAMES
Platform	Ethereum / Solidity
Methods	Architecture Review, Functional Testing, Computer-Aided Verification, Manual Review
Archive Name	contracts.zip
SHA256 Checksum	4209E9404B56508A1DD118BEEBA621952E6AD7A230673499061B147489749DB8
Timeline	10 TH SEP 2020 - 11 TH SEP 2020
Changelog	11 TH SEP 2020 - Initial Audit

Table of contents

Document.....	2
Table of contents.....	3
Introduction.....	4
Scope.....	4
Executive Summary.....	5
Severity Definitions.....	6
AS-IS overview.....	7
Conclusion.....	23
Disclaimers.....	24

Introduction

Hacken OÜ (Consultant) was contracted by CHAIN GAMES (Customer) to conduct a Smart Contract Code Review and Security Analysis. This report presents the findings of the security assessment of Customer's smart contract and its code review conducted between September 10, 2020 - September 11, 2020.

Scope

The scope of the project is smart contracts in the repository:

Archive Name - contracts.zip

SHA256 Checksum -

4209E9404B56508A1DD118BEEBA621952E6AD7A230673499061B147489749DB8

./contracts/BurnableERC20.sol	Out of Scope of Review
./contracts/ChainBetManager.sol	In Scope of Review
./contracts/ChainStakingContract.sol	Out of Scope of Review
./contracts/ChainToken.sol	Out of Scope of Review

We have scanned this smart contract for commonly known and more specific vulnerabilities. Here are some of the commonly known vulnerabilities that are considered:

Category	Check Item
Code review	- Reentrancy - Ownership Takeover - Timestamp Dependence - Gas Limit and Loops - DoS with (Unexpected) Throw - DoS with Block Gas Limit - Transaction-Ordering Dependence - Style guide violation - Costly Loop - ERC20 API violation - Unchecked external call - Unchecked math - Unsafe type inference - Implicit visibility level - Deployment Consistency - Repository Consistency - Data Consistency

Functional review	▪ Business Logics Review ▪ Functionality Checks ▪ Access Control & Authorization ▪ Escrow manipulation ▪ Token Supply manipulation ▪ User Balances manipulation ▪ Data Consistency manipulation ▪ Kill-Switch Mechanism ▪ Operation Trails & Event Generation
-------------------	---

Executive Summary

According to the assessment, the Customer's smart contracts do not have high vulnerability and can be considered secure. Some fixes are needed though.

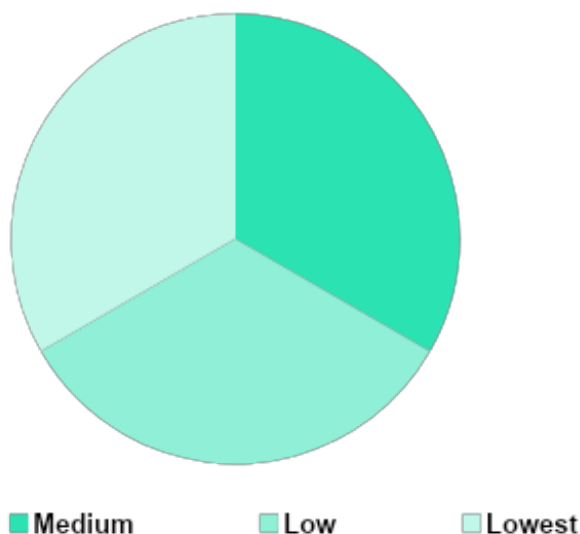
Insecure	Poor secured	Secured	We-secured
----------	--------------	---------	------------

 You are here

Our team performed an analysis of code functionality, manual audit, and automated checks with Mythril and Slither. All issues found during automated analysis were manually reviewed and important vulnerabilities are presented in the Audit overview section. A general overview is presented in AS-IS section and all found issues can be found in the Audit overview section.

Security engineers found 1 medium, 1 low and 1 lowest severity issues during audit.

Graph 1. The distribution of vulnerabilities.



Severity Definitions

Risk Level	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to assets lose or data manipulations.
High	High-level vulnerabilities are difficult to exploit; however, they also have significant impact on smart contract execution, e.g. public access to crucial functions
Medium	Medium-level vulnerabilities are important to fix; however, they can't lead to assets lose or data manipulations.
Low	Low-level vulnerabilities are mostly related to outdated, unused etc. code snippets, that can't have significant impact on execution
Lowest / Code Style / Best Practice	Lowest-level vulnerabilities, code style violations and info statements can't affect smart contract execution and can be ignored.

AS-IS overview

ChainBetManager.sol

Description

ChainBetManager contract allows to manage bets placed in CHAIN games.

Imports

ChainBetManager contract has 4 imports:

- *AccessControl.sol* – from OpenZeppelin;
- *ReentrancyGuard.sol* – from OpenZeppelin;
- *Pausable.sol* – from OpenZeppelin;
- *BurnableERC20.sol* – from project files;

Inheritance

ChainBetManager contract inherits *Initializable*, *ContextUpgradeSafe*, *AccessControlUpgradeSafe*, *PausableUpgradeSafe*.

Structs

ChainBetManager contract has 3 data structures:

- *LobbyInfo* struct has 5 fields:
 - o *address creator* – an address of creator;
 - o *uint256 bet* – a lobby bet;
 - o *uint32 currentMatch* – a match id;
 - o *uint8 gID* – a game id;
 - o *bool exists* – a lobby existence indicator
- *MatchInfo* struct has 4 fields:
 - o *uint256 pool* – a pool of the match;
 - o *uint32 lobbyID* – a lobby id;
 - o *bool exists* – a match existence indicator;
 - o *mapping (address=>bool) players* – a list of match players;
- *Winner* struct has 2 fields:
 - o *address player* – an address of winner;
 - o *uint256 winnings* – a prize for the winner;

Fields

ChainBetManager contract has 13 fields:

- *bytes32 public constant PAUSER_ROLE* – a pauser role indicator;
- *bytes32 public constant MANAGER_ROLE* – a manager role indicator;
- *uint256[3] internal shares3Winners* – represents the distribution of the reward among 3 winners
- *uint256[2] internal shares2Winners* – represents the distribution of the reward among 2 winners
- *uint256 internal feesHeld* – all collected fees;
- *uint256 internal globalPool* – a global pool;
- *address payable public companyWallet* – an address of the company;
- *address payable public stakingRewardsAddress* – an address where rewards stored;
- *uint32 internal lobbyCount* – an ever-incrementing counter used to generate unique lobby IDs;
- *uint32 internal matchCount* – an ever-incrementing counter used to generate unique match IDs
- *BurnableERC20 public token* – burnable ERC-20 token;
- *mapping (uint32=>LobbyInfo) public lobbies* – a list of lobbies;
- *mapping (uint32=>MatchInfo) public matches* – a list of matches;

Modifiers

ChainBetManager contract has 3 modifiers:

- *onlyContract(address account)* – checks if *account* is a contract address;
- *onlyManager()* – checks if caller has Manager role;
- *onlyAdmin()* – checks if caller has Admin role;

Functions

ChainBetManager has 22 functions:

- ***initialize***

Description

Initializes contract.

Visibility

public

Input parameters

- o *address _token* – an address of token contract;
- o *address payable _stakingRewardsAddress* – an address to store rewards;

Constraints

None

Events emit

None

Output

None

● ***__ChainBetManager_init***

Description

Initializes contract.

Sets *token*, *companyWallet*, *stakingRewardsAddress*, *shares3Winners*, *shares2Winners*.

Visibility

internal initializer

Input parameters

- o *address _token* – an address of token contract;
- o *address payable _stakingRewardsAddress* – an address to store rewards;

Constraints

- o *_token* must be the address of the contract;

Events emit

None

Output

None

- ***__ChainBetManager_init_unchained***

Description

Sets default roles.

Visibility

internal initializer

Input parameters

None

Constraints

None

Events emit

None

Output

None

- ***pause***

Description

Pauses all betting.

Visibility

public

Input parameters

None

Constraints

- o A caller must have the *PAUSER_ROLE*.

Events emit

None

Output

None

- ***unpause***

Description

Unpauses all betting.

Visibility

public

Input parameters

None

Constraints

- o A caller must have the *PAUSER_ROLE*.

Events emit

None

Output

None

- ***version***

Description

Returns a contract version.

Visibility

public pure

Input parameters

None

Constraints

None

Events emit

None

Output

Returns a contract version.

● *createLobby*

Description

Creates a new lobby.

Visibility

external

Input parameters

- o *uint8 gameId* – a game id;
- o *uint256 bet* – a bet;

Constraints

- o When betting not paused;
- o A caller must be a Manager;

Events emit

- o *emit LobbyCreated(id);*

Output

None

● *_createMatch*

Description

This document is proprietary and confidential. No part of this document may be disclosed in any manner to a third party without the prior written consent of Hacken.

www.hacken.io

Creates the match.

Visibility

internal

Input parameters

- o *uint32 lobbyID* – a lobby id;

Constraints

None

Events emit

None

Output

Match id.

● *addPlayer*

Description

Used to add player to lobby.

Visibility

external

Input parameters

None

Constraints

- o When betting not paused;
- o A caller must be a Manager;
- o Lobby exists;
- o The player is not in the lobby yet;
- o The player paid the bet.

Events emit

None

Output

None

● *_addPlayersToMatch*

Description

Used to add players to match.

Visibility

internal

Input parameters

- o *MatchInfo storage mInfo* – a match info;
- o *uint256 betValue* – a bet value;
- o *address[] memory players* – an array of players;

Constraints

None

Events emit

- o *emit PlayersAdded(errorStrings);*

Output

None

● *declareWinners*

Description

Distributes the reward to the winners of the current lobby match.

Visibility

external

Input parameters

- o *uint32 lobbyID* – a lobby id;

- o *address[] calldata winners* – an array of winners;

Constraints

- o When betting not paused;
- o A caller must be a Manager;
- o There can be no more than 3 winners;
- o Lobby exists;

Events emit

- o *emit WinnersDeclared(lobbyID, fee, detailsOfWinners);*

Output

None

● *updateLobbyBet*

Description

Updates bet in the lobby.

Visibility

external

Input parameters

- o *uint32 lobbyID* – a lobby id;
- o *uint256 newBet* – a new bet;

Constraints

- o A caller must be a Manager;
- o Lobby exists;
- o The lobby has no players in it;
- o A new bet must be greater than 0;

Events emit

None

Output

None

- ***updateBetAndAddPlayers***

Description

Updates bet in the lobby and add players to it.

Visibility

external

Input parameters

- o *uint32 lobbyID* – a lobby id;
- o *uint256 newBet* – a new bet;
- o *address[] calldata players* – an array of players;

Constraints

- o When betting not paused;
- o A caller must be a Manager;
- o The lobby exists;
- o The lobby has no players in it;
- o A new bet must be greater than 0;

Events emit

None

Output

None

- ***addPlayersToLobby***

Description

Used to add players to lobby.

Visibility

external

Input parameters

- o *address[] calldata players* – an array of players;
- o *uint32 lobbyID* – a lobby id;

Constraints

- o When betting not paused;
- o A caller must be a Manager;
- o The lobby exists;

Events emit

None

Output

None

● *calculateFee*

Description

Calculates fee.

Visibility

internal pure

Input parameters

- o *uint256 total* – an amount for which the fee is calculated;

Constraints

None

Events emit

None

Output

Returns fee.

● *setCompanyWallet*

Description

Sets a company wallet address.

Visibility

external

Input parameters

- o *address payable newAddress* – an address;

Constraints

- o A caller must be an Admin;
- o *newAddress* can't be the zero address;
- o New address should not be such as old;

Events emit

None

Output

None

● *setStakingRewardsAddress*

Description

Sets a staking rewards address.

Visibility

external

Input parameters

- o *address payable newAddress* – an address;

Constraints

- o A caller must be an Admin;
- o *newAddress* can't be the zero address;
- o New address should not be such as old;

Events emit

None

Output

None

- ***collectFees***

Description

Distributes fee tokens by wallets.

Visibility

external

Input parameters

None

Constraints

- o A caller must be an Admin;

Events emit

None

Output

None

- ***withdrawToken***

Description

Used to transfer non-CHAIN tokens accidentally sent to the contract.

Visibility

external

Input parameters

- o *address tokenAddress* – an address of non-CHAIN token contract;
- o *address destination* – an address of tokens receiver;
- o *uint256 value* – an amount of tokens to withdraw;

Constraints

- o A caller must be an Admin;
- o Can't withdraw CHAIN tokens;

Events emit

None

Output

None

● *withdrawUnaccountedTokens*

Description

Used to transfer unaccounted CHAIN tokens held by this contract. In case someone sent them by mistake to this contract.

Visibility

external

Input parameters

- o *address destination* – an address of tokens receiver;
- o *uint256 value* – an amount of tokens to withdraw;

Constraints

- o A caller must be an Admin;
- o Can't withdraw more tokens than have;

Events emit

None

Output

None

● *unaccountedTokens*

Description

Returns CHAIN tokens held by this contract that are not counted.

Visibility

external view

Input parameters

None

Constraints

- o A caller must be an Admin;

Events emit

None

Output

Returns an amount of tokens.

● ***_unaccountedTokens***

Description

Returns CHAIN tokens held by this contract that are not counted.

Visibility

internal view

Input parameters

None

Constraints

- o The actual balance should be higher than expected;

Events emit

None

Output

Returns an amount of tokens.

Audit overview

■ ■ ■ ■ Critical

No critical issues were found.

■ ■ ■ High

No high severity issues were found.

■ ■ Medium

1. It strongly recommended using *SafeMath* in calculations to prevent overflows.

■ Low

1. The *_addPlayersToMatch* function has a loop that does not have a fixed number of iterations. This can lead to exceeding the gas limit. It is recommended to limit the maximum number of players.

■ Lowest / Code style / Best Practice

1. The code for the *updateLobbyBet* function is used in the *updateBetAndAddPlayers* function, so it is better to create an internal *_updateLobbyBet* function and call it, rather than copy the code.

Conclusion

Smart contracts within the scope was manually reviewed and analyzed with static analysis tools. For the contract high level description of functionality was presented in As-is overview section of the report.

Audit report contains all found security vulnerabilities and other issues in the reviewed code.

Violations in following categories were found and addressed to Customer:

Category	Check Item	Comments
Code review	▪ Gas Limit and Loops	Loops that do not have a fixed number of iterations, for example, loops that depend on storage values, have to be used carefully.
	▪ Unchecked math	It strongly recommended using <i>SafeMath</i> in calculations to prevent overflows.
	▪ Style guide violation	If a piece of code is used more times, then it is better to move it into a function and call it.

Security engineers found 1 medium, 1 low and 1 lowest severity issues during audit. It is recommended to fix them.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment and functionality (performing the intended functions).

The audit makes no statements or warranties on security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bugfree status or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only - we recommend proceeding with several independent audits and a public bug bounty program to ensure security of smart contracts.

Technical Disclaimer

Smart contracts are deployed and executed on blockchain platform. The platform, its programming language, and other software related to the smart contract can have own vulnerabilities that can lead to hacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.